Introduction to Java

I. Understanding Java: What and Why?

1. What Is Java?

When we begin learning programming, one question naturally appears: *Why does Java remain one of the most popular programming languages even after decades?* The answer lies in its philosophy — **write once, run anywhere**.

Java is a **high-level, object-oriented programming language** developed by Sun Microsystems in 1995 (now owned by Oracle). It was designed to be simple, secure, portable, and powerful enough to build everything from desktop applications to enterprise systems and Android apps.

Unlike languages tied closely to hardware, Java acts like a universal translator between humans and machines. We write code once, and Java ensures it works across different operating systems without rewriting it.

Think of Java as a standardized electrical plug. No matter where you travel, adapters help it work everywhere — Java's adapter is called the **Java Virtual Machine (JVM)**.

2. Why Do We Use Java?

We use Java because it solves real-world programming challenges efficiently. Its main advantages include:

- Platform independence
- Strong memory management
- Built-in security features
- Large community support
- Rich libraries and frameworks

Java is widely used in:

- Enterprise software systems
- Android mobile applications
- Banking and financial platforms
- Web applications
- Cloud-based systems

If programming languages were vehicles, Java would be a reliable SUV — not flashy, but extremely dependable.

II. Key Features of Java

1. Object-Oriented Nature

Java follows **Object-Oriented Programming (OOP)** principles. Instead of thinking only in instructions, we think in terms of **objects** — real-world entities with properties and behaviors.

For example:

- A *Car* has attributes (color, speed).
- A *Car* performs actions (drive, brake).

Java organizes programs using:

- Classes
- Objects
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

This structure makes programs easier to manage and reuse.

2. Platform Independence

Java programs do not run directly on the operating system. Instead:

1. Source code → compiled into **bytecode**
2. Bytecode → executed by JVM

Because every system has its own JVM, the same program runs anywhere.

We often summarize this as:

Write Once, Run Anywhere (WORA)

3. Security

Java was built with internet applications in mind, so security became a core feature. Java includes:

- Bytecode verification

- Secure class loading
- No direct memory access (no pointers)

This reduces system crashes and malicious behavior.

4. Robustness and Reliability

Java avoids many common programming errors through:

- Strong type checking
- Automatic garbage collection
- Exception handling

Instead of manually cleaning memory, Java automatically removes unused objects — like a self-cleaning workspace.

5. Multithreading Capability

Java allows multiple tasks to run simultaneously using **threads**.
Imagine downloading a file while listening to music. Java programs can perform multiple operations without freezing — essential for modern applications.

III. Java Architecture and Execution Process

1. Java Development Kit (JDK)

The JDK is the complete toolbox for Java developers. It includes:

- Compiler (`javac`)
- JVM
- Libraries
- Development tools

Without JDK, we cannot build Java programs.

2. Java Runtime Environment (JRE)

The JRE provides everything needed to **run** Java programs but not develop them.
It contains:

- JVM
- Core libraries

In simple words:

- **JDK → Create programs**
- **JRE → Run programs**

3. Java Virtual Machine (JVM)

The JVM is the heart of Java.
Its responsibilities include:

- Executing bytecode
- Managing memory
- Handling garbage collection
- Providing platform independence

We can imagine JVM as a virtual computer inside our real computer.

4. Compilation and Execution Flow

Let us walk through the lifecycle of a Java program:

1. Write program → `.java` file
2. Compile using `javac` → `.class` bytecode
3. JVM executes bytecode

This two-step execution makes Java portable.

IV. Basic Structure of a Java Program

1. The Simplest Java Program

A typical Java program looks like this:

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("Hello, World!");
}
}
```

At first glance, it may look complex, but each part has a purpose.

2. Understanding Each Component

Class Declaration

```
public class HelloWorld
```

Every Java program must belong to a class.

Main Method

```
public static void main(String[] args)
```

This is the starting point of execution. Without it, Java does not know where to begin.

Output Statement

```
System.out.println();
```

This prints output on the console.
Think of the `main` method as the front door of a house — Java always enters through it.

3. Rules of Java Programming

Java follows strict syntax rules:

- File name must match class name.
- Java is case-sensitive.
- Every statement ends with `;`.
- Code blocks use `{ }`.

These rules may feel strict, but they prevent confusion later.

V. Variables, Data Types, and Operators

1. Variables

A variable stores data.
Example:

```
int age = 20;
```

Here:

- `int` → data type
- `age` → variable name
- `20` → value

Variables are like labeled containers storing information.

2. Data Types in Java

Java provides several data types:

Primitive Types

- `int` – integers
- `float` – decimal numbers
- `double` – large decimals
- `char` – characters
- `boolean` – true/false

Non-Primitive Types

- String
- Arrays
- Classes
- Objects

Choosing the correct data type improves performance and clarity.

3. Operators

Operators perform operations on data.

Arithmetic Operators

```
+ - * / %
```

Relational Operators

```
== != > <
```

Logical Operators

```
&& || !
```

Operators allow programs to make decisions and calculations — essentially giving logic to code.

VI. Control Statements in Java

Programs become powerful when they can make decisions.

1. Conditional Statements

if Statement

```
if(age >= 18){
System.out.println("Adult");
}
```

Used when actions depend on conditions.

if-else Statement
Allows alternative execution paths.

switch Statement
Useful when selecting from multiple choices.

2. Looping Statements

Loops repeat tasks automatically.
for Loop

```
for(int i=0; i<5; i++){
System.out.println(i);
}
```

while Loop Runs while a condition remains true.
do-while Loop Executes at least once before checking condition.
Loops save us from writing repetitive code — like automation in real life.

VII. Introduction to Object-Oriented Programming in Java

1. Classes and Objects

A **class** is a blueprint.An **object** is a real instance.
Example:

```
class Student {
String name;
}
```

Creating object:

```
Student s = new Student();
```

2. Encapsulation

Encapsulation protects data using private variables and public methods.
It is similar to using an ATM — we interact through buttons without seeing internal mechanisms.

3. Inheritance

Inheritance allows one class to reuse properties of another.
Example:

```
Animal → Dog
```

Dog automatically gains features of Animal.
This reduces repetition and improves code reuse.

4. Polymorphism

One action, many forms.
Example:A method named `draw()` can behave differently for Circle, Square, or Triangle.

5. Abstraction

Abstraction hides complexity and shows only necessary details.
When driving a car, we use the steering wheel without knowing engine mechanics — that is abstraction.

VIII. Applications of Java in the Real World

Java powers many technologies we use daily:

- Android applications
- Banking transaction systems
- E-commerce platforms
- Web servers
- Big data technologies

Companies rely on Java because it scales well and remains stable over time.

IX. Advantages and Limitations of Java

Advantages

- Platform independent
- Secure and robust
- Object-oriented
- Huge ecosystem
- Automatic memory management

Limitations

- Slightly slower than low-level languages
- Requires more memory
- Verbose syntax

Yet, for large systems, reliability matters more than small speed differences.

Conclusion

Java is more than just a programming language; it is a structured way of thinking about software development. Throughout this lecture, we explored its origins, architecture, features, syntax, and object-oriented foundations. We saw how Java transforms human logic into machine instructions while maintaining portability, security, and scalability. As we move forward, mastering Java means not only learning syntax but also understanding problem-solving through objects, logic, and structured design. Once we grasp these fundamentals, Java becomes less of a language and more of a powerful toolkit — one that allows us to build reliable systems that can grow, adapt, and endure in an ever-evolving technological world.